Teoría de los Lenguajes de Programación Práctica curso 2025-2026

Enunciado

Equipo Docente de la Asignatura

1. Futón chiqui… digooo… Futoshiki, ese puzzle impronunciable.

En la empresa "Tecleamos Los Programas" (TLP) tuvo muchísimo éxito el concurso de Mastermind/Wordle del curso anterior, pero el tiempo ha pasado, el entusiasmo de la gente se fue desinflando y ya no es lo mismo. Ahora los empleados van a la empresa… ¡a trabajar! ¡Dónde se ha visto eso! Ya no se divierten tanto y los semblantes son cada vez más sombríos.

Los jefes vuelven a temblar recordando lo que significa que los empleados no estén contentos y motivados: disminución de la productividad y aumento exponencial de errores lógicos en los programas. Y, lógicamente, eso no lo pueden permitir. La verdad es que están empezando a sospechar que lo que realmente motiva a los empleados no son los juegos en sí, sino los retos de resolverlos.

Así pues se han puesto a buscar un puzzle sencillo de entender, pero que represente un reto para ser resuelto. ¿Sudoku? No, eso ya lo saben resolver. ¿Hashiwokakero? Es complicado (además de pronunciar) de explicar en qué consiste. ¿Nonogramas? No, no… ese también está resuelto.

Y entonces alguien dijo algo sobre un futón pequeño… un futón chiqui… ¡El Futoshiki! ¡Ya lo tenemos! ¡No se hable más! (más que nada porque cada uno pronuncia el puzzle de una forma diferente)

Por lo tanto, como expertos en resolución de puzzles (y, además, fervientes defensores de la programación en Haskell), nuestra tarea va a ser realizar un programa que resuelva estos puzzles (se pronuncien como se pronuncien).

¡Tenemos nuevamente en nuestras manos la salvación de la empresa! ¿Aceptaremos el reto?

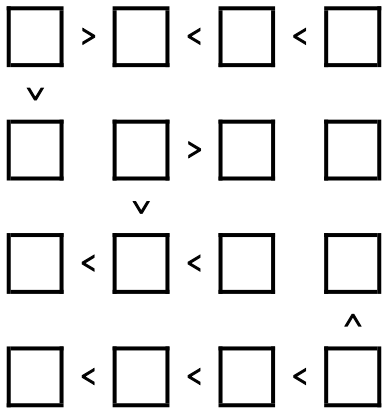
2. Enunciado de la práctica

La práctica consiste en elaborar un programa en **Haskell** que reciba un puzzle Futoshiki y devuelva una lista de soluciones del mismo.

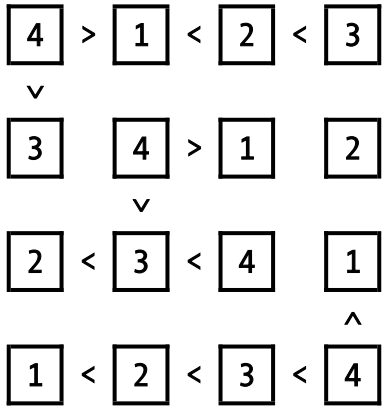
El Futoshiki (también conocido como "Más o Menos") es un puzzle lógico de origen japonés, desarrollado por Tamaki Seto en 2001. Se juega en un tablero cuadrado de NxN y consiste en colocar los números del 1 al N en las casillas vacías, respetando las siguientes reglas:

1. En una misma fila no puede haber números repetidos.
2. En una misma columna no puede haber números repetidos.
3. Algunas casillas contiguas (tanto vertical como horizontalmente) presentan restricciones adicionales de orden entre sus elementos, indicando cuál de ellos debe ser mayor que el otro.

Vamos a ver un ejemplo y comenzaremos por el siguiente puzzle inicial:



Vemos que todas las casillas están vacías y que entre algunas de ellas (tanto vertical como horizontalmente) hay símbolos de mayor o menor. Eso significa que los números en esas casillas deben respetar esa relación. La solución (porque para este puzzle solo hay una) es la siguiente:



Por lo tanto, la práctica consiste en programar una función `solve`, que recibirá un Futoshiki y devolverá una lista con todas las soluciones del puzzle.

2.1 Estructura de módulos de la práctica

La práctica está dividida en dos módulos, cada uno en un fichero independiente cuyo nombre coincide (incluyendo mayúsculas y minúsculas) con el nombre del módulo y cuya extensión es .hs:

1. Módulo `Futoshiki`: dentro de este módulo se programará las función solve. Incluye las definiciones de todos los tipos de datos utilizados en la práctica.
2. Módulo `Main`: este módulo contiene la función principal y las encargadas de interactuar con el usuario. Este módulo se proporciona ya totalmente programado y no deberá modificarse.

Dado que un estudio más profundo de los módulos en **Haskell** está fuera del ámbito de la asignatura, sólo indicaremos que el módulo `Main` importa (además del módulo `Futoshiki`) algunos módulos adicionales para poder trabajar más cómodamente con la entrada/salida.

2.1.1 Módulo Main (programa principal)

El módulo `Main` contiene el programa principal, ya programado por el equipo docente. Además del módulo Futoshiki, también importa una serie de módulos adicionales para gestionar la entrada/salida y calcular el tiempo de ejecución.

Algunas de las funciones utilizan mónadas y están escritas utilizando la "*notación do*", que es una notación para facilitar la escritura de concatenaciones de funciones monádicas. Sin entrar en detalle sobre el funcionamiento de las mónadas, vamos a explicar qué hace cada función:

- `readTests`: esta función lee un fichero de tests y devuelve el contenido del mismo en forma de lista de cadenas (una cadena por cada línea del fichero). En caso de error, devuelve una lista vacía
- `readMaybeFile`: esta función intenta leer un fichero y devuelve una mónada `Maybe`, que puede representar una lectura correcta (devolviendo el contenido) o un error.
- `executeTests`: recibe una lista de tests (cada uno codificado en forma de `String`) y ejecuta todos ellos llamando a la función `checkTest`.
- `checkTest`: recibe un test en forma de `String`, lo intenta convertir al tipo `TestFutoshiki` y en caso de éxito llama a la función `checkSolution` con el puzzle y la solución esperada. Además, muestra el tiempo empleado (en milisegundos) en la resolución del puzzle.

- `checkSolution`: recibe un `Futoshiki` y la solución esperada y comprueba si la función `solve` devuelve un resultado acorde al esperado.
- `loop`: es el "bucle" principal del programa, que pide al usuario el nombre del fichero de tests a ser cargado, lo lee y llama a la función `executeTests`.
- `main`: es la función principal. Configura la entrada para que se pueda editar (si no, no tendría efecto la tecla de retroceso) y llama a la función `loop`.

Como ya se ha indicado, este módulo se entrega ya programado por el equipo docente. Para su correcto funcionamiento hay que programar la función `solve` en el módulo `Futoshiki`.

2.1.2 Módulo Futoshiki

En este módulo se debe programar la función `solve` y cualesquiera funciones auxiliares que sean necesarias. En él se incluyen las definiciones de los tipos de datos necesarios para representar un Futoshiki:

```
type Matrix a = [[a]]

data Relation = Ind | RGT | RLT
  deriving (Eq, Read, Show)

data Futoshiki = Futoshiki {
  size  :: Int,
  cells :: Matrix Int,
  hRel  :: Matrix Relation,
  vRel  :: Matrix Relation
}
  deriving (Eq, Read, Show)
```

Los tipos de datos son las siguientes:

1. `Matrix a`: almacena una matriz bidimensional de elementos de cualquier tipo.
2. `Relation`: almacena las relaciones entre dos casillas adyacentes. Sus valores adquieren el siguiente significado:
   - `Ind`: los valores de las dos casillas son **Ind**ependientes. No tienen ninguna relación.
   - `RGT`: los valores de las dos casillas están **R**elacionados, siendo la primera más grande (**G**reater **T**han) la segunda.
   - `RLT`: los valores de las dos casillas están **R**elacionados, siendo la primera más pequeña (**L**ess **T**han) la segunda.

   Seguiremos la convención de que entre dos casillas adyacentes horizontalmente, la primera será la de la izquierda y la segunda la de la derecha. Si son adyacentes verticalmente, la primera casilla será la superior y la segunda la inferior.

   Además, este tipo se instancia en las clases `Eq`, `Read` y `Show`, lo que permite compararlo, leerlo desde un `String` y mostrarlo.
3. `Futoshiki`: es un tipo que nos permite almacenar el estado de un puzzle Futoshiki. Al utilizar la *notación de registro*, Haskell nos crea automáticamente unas funciones que nos permiten acceder a las componentes del tipo. Así, dado un elemento f de tipo Futoshiki:
   - `size f` nos devuelve el tamaño del puzzle
   - `cells f` nos devuelve la matriz de números del puzzle
   - `hRel f` nos devuelve la matriz de relaciones horizontales del puzzle
   - `vRel f` nos devuelve la matriz de relaciones verticales del puzzle

En la matriz de números un 0 representa una casilla vacía, por lo que el estado inicial del puzzle que hemos utilizado como ejemplo se codificaría como sigue:

```
Futoshiki {size = 4,
           cells = [[0,0,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]],
           hRel = [[RGT,RLT,RLT],
                   [Ind,RGT,Ind],
                   [RLT,RLT,Ind],
                   [RLT,RLT,RLT]],
           vRel = [[RGT,Ind,Ind,Ind],
                   [Ind,RGT,Ind,Ind],
                   [Ind,Ind,Ind,RLT]]
          }
```

mientras que la solución del mismo tendría la siguiente codificación:

```
Futoshiki {size = 4,
           cells = [[4,1,2,3],[3,4,1,2],[2,3,4,1],[1,2,3,4]],
           hRel = [[RGT,RLT,RLT],
                   [Ind,RGT,Ind],
                   [RLT,RLT,Ind],
                   [RLT,RLT,RLT]],
           vRel = [[RGT,Ind,Ind,Ind],
                   [Ind,RGT,Ind,Ind],
                   [Ind,Ind,Ind,RLT]]
          }
```

Este tipo también se instancia en las clases `Eq`, `Read` y `Show`, lo que permite compararlo, leerlo de un `String` y mostrarlo.

Además, el módulo Futoshiki también incluye la función `bt`, que implementa el esquema de backtracking.

2.2 Método de resolución de la práctica

Para resolver la práctica vamos a utilizar el algoritmo de backtracking. Cada nodo del árbol de backtracking será un elemento del tipo `Futoshiki`. La raíz del árbol será el problema inicial y a cada nivel rellenaremos una casilla vacía más.

Por lo tanto, un nodo contendrá una solución si:

- Los números situados en las casillas cumplen las restricciones del enunciado…
- … y no quedan casillas vacías que rellenar.

Para poder aplicar la función genérica de backtracking, es necesario programar dos funciones auxiliares:

1. Una función que dado un nodo (es decir, un elemento de tipo `Futoshiki`) nos diga si ya es una solución o no.
2. Una función que dado un nodo nos devuelva su lista de nodos hijos.

Asumiendo que **partimos de un nodo que cumple todas las restricciones para las casillas que no estén vacías**, lo más adecuado será que los hijos de este nodo cumplan también todas las restricciones.

Esto reducirá el espacio de búsqueda y también simplificará la función que comprueba si un nodo es una solución, ya que por construcción, el nodo ya cumplirá las restricciones.

3. Evaluación de la práctica

La corrección del código de la práctica se evaluará contra un fichero de juego de pruebas al que solo tendrán acceso los/as tutores/as de la asignatura. Si el programa supera este juego de pruebas, la práctica estará calificada con un 5, a falta de evaluar las respuestas a las cuestiones sobre la práctica (ver apartado siguiente de este enunciado). Si alguno de los tests no es superado, la práctica se considerará suspensa.

Junto a este enunciado y al código programado de la práctica, se puede encontrar también un fichero con varios puzzles, que servirán a modo de juego de pruebas para estudiantes. A lo largo del cuatrimestre animamos a los y las estudiantes a compartir nuevos puzzles en el foro sobre la práctica de la asignatura para crear más ficheros de prueba.

4. Cuestiones sobre la práctica

La respuesta a estas preguntas es optativa. Sin embargo, si no se responde a estas preguntas, la calificación de la práctica solo podrá llegar a 5 puntos sobre 10.

1 Supongamos una implementación de la práctica (usando los mismos tipos de datos aquí presentados) en un lenguaje no declarativo (como **Java**, **Pascal**, **C**…). Entonces, comente qué ventajas y desventajas tendría frente a la implementación en Haskell en relación a lo siguiente:

   a) (*1 punto*). Eficiencia con respecto a la programación.
   b) (1 punto). Eficiencia con respecto a la ejecución.
   c) (1 *punto*). Con respecto a los dos tipos de eficiencia anteriores, ¿qué paradigma (declarativo vs. no declarativo) sale beneficiado gracias a la semántica de sus variables?
   d) (*0,5 puntos*). ¿Cuál sería el principal punto a favor de la implementación en Haskell?

   Razone y justifique sus respuestas.

2 (*1,5 puntos*). Indique qué **clases de constructores de tipos** (ver capítulo 5 del libro de la asignatura) se han utilizado para definir los tipos de datos presentes en el módulo `Futoshiki`. Justifique sus respuestas.

5. Documentación a entregar

Cada estudiante deberá entregar la siguiente documentación a su tutor/a de prácticas:

- Código fuente en **Haskell** que resuelva el problema planteado. Para ello se deberá entregar el ficheros `Futoshiki.hs`, con las funciones descritas en este enunciado, así como todas las funciones auxiliares que sean necesarias.
- Una memoria con las respuestas a las cuestiones sobre la práctica.